Introduction

Important: make sure to read the specification for this project phase to the very end!

As you go through each phase of the term project, it's important to keep in mind the information presented in the following sections:

- overview
- description of the project structure

Phase 3 of the term project is worth 30% of your overall project grade.

Academic Honesty

You are reminded that all the code that you submit at each phase of the term project must be

- code that you have written yourself since the start of this term (you must not make use of any code that you have written in the past)
- code found in or based on any of the projects that we present to you in lecture during the term, provided you insert a comment into your code to attribute the source

Please ensure that you do not provide other students with access to your code. By doing so, you risk enabling academic misconduct for which you will be held accountable. Typically, the same grade penalty is applied to the student who enabled access to their code, as to the student who used the code.

If you find the project challenging, ask for help from a course instructor or from one of the TAs *assigned to the course this term.* Do not be tempted to copy code from someone else and do not, under any circumstances, allow another student access to your code in any way.

General Requirements

You should commit and push your code to GitHub after completion of each task, if not more frequently. By pushing your code to GitHub you are effectively making a backup of your work which you'll be able to retrieve if anything happens to the code on your laptop.

All code must be appropriately documented:

- each class should have a class-level comment (that appears just before the class declaration) to describe the information that the class represents
- every method must be documented with requires, modifies and effects clauses, as appropriate (note that simple setter and getter methods, and test methods, do not have to be documented)

You code must adhere to our style guidelines. Note that for the term project, the maximum method length has been increased from 20 to 25 lines.

That said, there may be some cases where it does not make sense to decompose a method whose length is greater than 25 lines. In such cases, you may include an `@SuppressWarnings("methodlength")` annotation on the line immediately preceding the method header. This instructs checkstyle not to apply the method length restriction to that particular method. You should always check with your TA before using this annotation, as any mis-use will result in a grade penalty. Note that AutoTest will list all the places in your code where the annotation is used.

Main Tasks

In Phase 3 and, indeed, all future phases of the term project, you will be working with the same repository that was provided to you for Phase 1.

This phase of the term project involves a significant amount of work. **You are expected to make steady progress and commit working code to your repository each week.** The previous statement will be taken into account when evaluating requests for extensions due to technical problems or other life events.

You will need to do some research and learn how to develop a Java Swing graphical user interface on your own. This is a topic that we do not teach in the video or in lecture but you have all the background and tools that you need to succeed in this task. It is important for any software engineer to attain the skills necessary to successfully use a new library or framework. We realize that this task is challenging, so your TAs and instructors will be available to help out if you get stuck. However, there's a limit to how much help we can provide if everyone leaves it to just 2 or 3 days before the deadline. We've given you a significant amount of time to complete this task - please use it wisely.

Task 1

If you have not done so already, please ensure that you have committed your Phase 2 code, pushed it to GitHub and requested a grade from AutoTest for your Phase 2 deliverables. We then have a record of the work that you completed during Phase 2, so that you may start work on Phase 3.

Task 2

In this phase of the term project, you are not required to add new user stories for your term project, although you may do so if you wish. (Read carefully below, however, to understand why you are not responsible for implementing all your user stories in Phase 3.) You are not allowed to remove any functionality from your model package. In other words, ideally, your old console UI should still work, your model should be unchanged, and your coverage score should remain high. If you must make changes to the model to get your GUI working, ensure that you carefully test any such changes. We may deduct marks if your model changes and your code coverage score drops.

If you choose to add any new user stories or add to any of the existing user stories, please commit and push the revised README.md file to GitHub.

Task 3

In this phase, you will need to do some research and learn how to develop a small graphical user interface (GUI) on your own using the Java Swing library (you must not use any other framework for developing your user interface).

All the code related to your GUI *must* be written in the ui package. As user interface testing is quite challenging, you are not required to design tests for any of the code directly related to your user interface.

You are not required to support all your user stories in your GUI but you are obviously free to do so, if you wish. However, you **must** provide support for the following user stories:

- "As a user, I want to be able to add multiple Xs to a Y", where X and Y are classes that you have designed as part of your application. Your GUI **must** include a panel in which all the Xs that have already been added to Y are displayed. You are also required to implement **two** related actions. For example, you could have a button or menu item that can be used to add an X to a Y, and another button or menu item that displays a subset of the Xs that satisfy some criterion specified by the user. Note that these are just examples - you are free to adapt these actions for your particular application but the actions that you implement **must** be somehow related to the Xs and Y.

application but the actions that you implement **must** be somehow related to the X's and Y's.

- "As a user, I want to be able to load and save the state of the application" OR "as a user, I wanted to be prompted with the *option* to load data from file when the application starts and prompted with the *option* to save data to file when the application ends". If you go with the first user story, you could have buttons or menu items that the user can click when they want to load/save data to/from file. If you go with the second user story, you could have pop-up windows that give the user the *option* (using Yes/No buttons) to load data when the application starts and to save it when the application ends.

Your GUI must also include a visual component, such as displaying an image when an action occurs, displaying an image in a splash screen when your application starts, or displaying a graph to represent some aspect of the user's data. Note that painting panels, buttons and menu items in different colours does not count. Please see the grading scheme below for further details.

As with Phase 1, it is a wonderful idea to design a clear, usable, attractive interface. However, we are not grading you on that. Your GUI MUST allow users to execute the user stories above, but it's OK if the interface isn't as attractive, clear, or usable as you would hope.

Suggestions

Start with one of the GUI projects you've already seen in CPSC210. SpaceInvaders, RobustTrafficLights, AlarmSystem, DrawingPlayer, and SmartHome all have different Swing GUIs that you could use as starting points.

If you want elements that aren't used in those projects, or just want to see what else is possible, visit:

https://docs.oracle.com/javase/tutorial/uiswing/examples/components/index.html

This tutorial includes zipped projects that you can download and run, most of which are fairly small and focus on just one or two GUI elements. You may want to use one of these demos as a starting point, or try to incorporate elements from them into your own project. If your project works with lists you'll find the list demos are perfect starting points.

Another fairly straightforward example can be found here: https://stackoverflow.com/questions/6578205/swing-jlabel-text-change-on-the-running-application. This application illustrates the very important concept of event handling. When the user clicks a button, moves or drags the mouse, or selects on option from a menu an *event* occurs to which we will want to respond. We do so by writing an event handler. We've copied the code below (changed the name of the class and added some comments, but otherwise it's from the stack overflow answer). Copy and paste it into a new Java application, and then run it:

```
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class LabelChanger extends JFrame implements ActionListener {
   private JLabel label;
   private JTextField field;

   public LabelChanger() {
      super("The title");
      setDefaultCloseOperation(EXIT_ON_CLOSE);
      setPreferredSize(new Dimension(400, 90));
      ((JPanel) getContentPane()).setBorder(new EmptyBorder(13, 13, 13, 13) );
      setLayout(new FlowLayout());
      JButton btn = new JButton("Change");
      btn.setActionCommand("myButton");
      btn.addActionListener(this); // Sets "this" object as an action listener for btn
                                   // so that when the btn is clicked,
                                   // this.actionPerformed(ActionEvent e) will be called.
                                   // You could also set a different object, if you wanted
                                   // a different object to respond to the button click
      label = new JLabel("flag");
      field = new JTextField(5);
      add(field);
      add(btn);
      add(label);
      pack();
      setLocationRelativeTo(null);
```

```
        setVisible(true);
        setResizable(false);
    }

    //This is the method that is called when the the JButton btn is clicked
    public void actionPerformed(ActionEvent e) {
        if (e.getActionCommand().equals("myButton")) {
            label.setText(field.getText());
        }
    }

    public static void main(String[] args) {
        new LabelChanger();
    }
}
```

You can find a more general introduction to event handling here:

https://docs.oracle.com/javase/tutorial/uiswing/events/intro.html

along with several examples of different kinds of events and their corresponding handlers. You may want to look through this tutorial for examples to illustrate the kinds of events that you want to handle in your project.

Documentation Requirements

Add a section at the end of your README.md file that includes the following:

- a new heading entitled "**Instructions for Grader**"
- a list of step-by-step instructions for interacting with your GUI that describes:
  - how to generate the first of the two required actions that are related to the required user story "add multiple Xs to a Y" (e.g., click the button labelled "Add an X", or select the menu item labelled "Add an X", or type <ctrl>+N on the keyboard to add an X,...)
  - how to generate the second of the two required actions
  - where to find the visual component that was added to your project (e.g., background image, image added to button)
  - how does the user save the state of the application to file
  - how does the user load the state of the application from file

Please use the following as a template. If you do not complete any of the bullet points below, we will assume that your project does not meet that requirement.

```
# Instructions for Grader

- You can generate the first required action related to adding Xs to a Y by...
- You can generate the second required action related to adding Xs to a Y by...
- You can locate my visual component by...
- You can save the state of my application by...
- You can reload the state of my application by...
```

Grading

When you are ready, commit and push your code to GitHub. Request feedback from AutoTest by making the following comment on your commit:

```
@210-bot #project
```

As you are all working on individual projects, AutoTest will obviously not give feedback on the correctness of your code. However, it will check that your code adheres to the CPSC 210 coding style. Although you are not required to test your user interface code, we continue to measure code coverage to ensure that you are testing any new

functionality that you choose to add to your project outside of the `ui` package.

Your grade for Phase 3 will be determined as follows:

- 50 pts - Recall that a required user story is that a user must be able to "add multiple Xs to a Y" where X and Y are classes that you've designed yourself. Your GUI must include a panel that displays the Xs that have been added to the Y. It must also allow the user to generate at least two actions related to those Xs and Y (e.g. filter the list of Xs to those of particular interest, highlight the most important X, remove an X, reorder the Xs, etc.) which are appropriately handled in the code.
- 30 pts - there are buttons or menu items that allow the user to save and to load the state of the application to/from file OR pop-up windows that are displayed when the application starts and when it ends giving the user the *option* to load/save data to/from file.
- 15 pts - in addition to the features identified above, the GUI must incorporate a visual component. For example, displays a graph of some aspect of the user's data, display a picture, etc. There are all kinds of possibilities!
- 5 pts - all methods added during Phase 3 are documented with appropriate requires/modifies/effects clauses.

Note:

- your code must pass checkstyle on AutoTest, otherwise it will not be graded
- you must get at least 50% on this phase of the term project. If you don't achieve that grade, you will have to continue to work on this phase of the term project and request a regrade for a maximum score of 50% on this phase of the term project. **Note that the deadline for Phase 4 is the very last opportunity to submit code for regrades on any of the earlier phases of the term project.**

< Previous Next >

© All Rights Reserved

edX Edge

edX

About
Affiliates
edX for Business
Open edX
Careers
News

Legal

Terms of Service & Honor Code
Privacy Policy
Accessibility Policy
Trademark Policy
Sitemap
Cookie Policy
Your Privacy Choices